

```

#include <stdio.h>
#define MAX 20
int main() {
    int block[MAX], process[MAX];
    int n, m, i, j;
    printf("Enter number of blocks: ");
    scanf("%d", &n);
    printf("Enter block sizes:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &block[i]);
    printf("Enter number of processes: ");
    scanf("%d", &m);
    printf("Enter process sizes:\n");
    for(i = 0; i < m; i++)
        scanf("%d", &process[i]);
    int temp[MAX];
    for(i = 0; i < n; i++) temp[i] = block[i];
    printf("\nFirst Fit:\n");
    for(i = 0; i < m; i++) {
        for(j = 0; j < n; j++) {
            if(temp[j] >= process[i]) {
                printf("P%d -> B%d\n", i+1, j+1);
                temp[j] -= process[i];
                break;
            }
        }
        if(j == n)
            printf("P%d -> Not Allocated\n", i+1);
    }
    printf("\nBest Fit:\n");
    for(i = 0; i < m; i++) {
        int best = -1;
        for(j = 0; j < n; j++) {
            if(temp[j] >= process[i]) {
                if(best == -1 || temp[j] < temp[best])
                    best = j;
            }
        }
        if(best != -1)
            printf("P%d -> B%d\n", i+1, best+1);
            temp[best] -= process[i];
    }
}

```

```

for(i = 0; i < n; i++) {
    int best = -1;
    for(j = 0; j < n; j++) {
        if(temp[j] >= process[i]) {
            if(best == -1 || temp[j] < temp[best])
                best = j;
        }
    }
    if(best != -1) {
        printf("P%d -> B%d\n", i+1, best+1);
        temp[best] -= process[i];
    } else {
        printf("P%d -> Not Allocated\n", i+1);
    }
}

for(i = 0; i < n; i++) temp[i] = block[i];
printf("\nWorst Fit:\n");
for(i = 0; i < n; i++) {
    int worst = -1;
    for(j = 0; j < n; j++) {
        if(temp[j] >= process[i]) {
            if(worst == -1 || temp[j] > temp[worst])
                worst = j;
        }
    }
    if(worst != -1) {
        printf("P%d -> B%d\n", i+1, worst+1);
        temp[worst] -= process[i];
    } else {
        printf("P%d -> Not Allocated\n", i+1);
    }
}

return 0;
}

```

Enter number of memory blocks: 5

Enter block sizes:

100 500 200 300 600

Enter number of processes: 4

Enter process sizes:

212 417 112 426

First Fit:

Process 1 -> Block 2

Process 2 -> Block 5

Process 3 -> Block 2

Process 4 -> Not Allocated

Best Fit:

Process 1 -> Block 4

Process 2 -> Block 2

Process 3 -> Block 3

Process 4 -> Block 5

Worst Fit:

Process 1 -> Block 5

Process 2 -> Block 2

Process 3 -> Block 5

Process 4 -> Not Allocated

Process exited after 55.73 seconds with return value 0

Press any key to continue . . . |